The purpose of this assignment is to test whether you meet the basic prerequisite for this class. You must complete the assignment independently, without getting help from other people. This assignment must be submitted to Canvas by **Jan 15, 11:59 PM Phoenix Time**. If you are enrolled into the class after Jan 13, please submit your assignment **within 2 days** (48 hours) after you are enrolled (No submissions will be allowed for late enrollments after Jan 19, 11:59 PM).

Objective

Create a small text dataset you will analyze later in the course. You will (1) collect metadata for songs you personally enjoy, and (2) normalize the lyrics text into a machine-readable form.

Data Collection & Preprocessing

- Collect 50 unique songs you personally enjoy, each released on or after 2025-01-01.
- For each song, you must provide both metadata and cleaned lyrics in the required order and format (below).
- Your file must be consistent and machine-parsable.

Rules and Guidelines

- **No duplicates:** Exactly 50 distinct songs (title + artist pair must be unique).
- **Time frame:** Release date **≥ 2025-01-01** (original release, not re-upload dates).
- **Accuracy:** Verify titles/artists/dates/genres from reputable sources (e.g., label site, MusicBrainz, Spotify/Apple Music metadata, Genius page).
- **Instrumentals:** If lyrics are effectively empty/very short, choose a different song.
- **Covers/remixes/feat.:**
    - Use the specific version you are citing.
    - **Artist** = credit as listed on the source (e.g., *Artist A feat. Artist B*).
    - **Title** = exact title including version tags if present (e.g., "Song remix").
- **Language:** Use the language of the lyrics (do **not** romanize non-Latin scripts). Mixed-language songs → pick the **primary** language listed by your source.
- **Ethics:** You may use publicly available lyrics from websites (e.g., Genius, AZLyrics, LyricsOnDemand, etc), but do not upload copyrighted audio files.

File Format (one line per song)

- **File type:** Plain text `.txt`, **UTF-8**, **no header line**.
- **Field delimiter:** `||` (two vertical bars). Do **not** put spaces around the delimiter.
- **Field order (exactly 8 fields):**
    - rank || title || artist || release_date || genre || language || lyrics_clean || source
    - Field Definitions:
        - `rank` — Your personal ranking for these 50 songs, with value 1 being your favorite.

- `title` — exact song title.
- `artist` — primary performer(s) exactly as credited.
- `release_date` — `YYYY-MM-DD`. (Find the official/label release date; do **not** approximate. If sources conflict, prefer the label/official site; otherwise pick the earliest reputable source and be consistent.)
- `genre` — Select from the predefined list.
  - Predefined list:
    - `pop, rock, hiphop, rb, country, jazz, edm, classical, kpop, latin, folk, metal, reggae, other`
  - If a song spans multiple genres, you may include more than one, separated by commas (no spaces).
    - Example: `pop,rock`.
  - If the genre does not match any listed category, use `other`.
  - Reference genre check:

```python
# Predefined genre list
VALID_GENRES = {
    "pop", "rock", "hiphop", "rb", "country", "jazz",
    "edm", "classical", "kpop", "latin", "folk",
    "metal", "reggae", "other"
}

def check_genre_field(genre_field: str) -> bool:
    """
    Check if all genres in a field are valid.
    Genres are separated by commas, no spaces allowed.
    """
    genres = genre_field.split(",")
    return all(g in VALID_GENRES for g in genres)

# Example usage:
examples = [
    "pop",              # ✅ valid
    "pop,rock",         # ✅ valid
    "latin,pop",        # ✅ valid
    "hip-hop",          # ❌ invalid (should be hiphop)
    "r&b",              # ❌ invalid (should be rb)
    "pop,indie"         # ❌ invalid (indie not in list)
]

for g in examples:
    print(g, "=>", check_genre_field(g))
```

- `language` — e.g., English, Spanish, Korean, Mandarin, Japanese, etc.

        - `lyrics_clean` — the **cleaned** lyrics text (see rules below).
          Important: `lyrics_clean` must **not** contain the delimiter `||` (our cleaning rules ensure this).
        - `source` — full website domain of the lyrics page (e.g., `https://genius.com/...`).
    - Example (One Entry):
        - 1||flowers||miley cyrus||2025-01-05||pop||english||we were good, we were gold, we were right til we werent. built a home and watched it burn ... ||https://www.azlyrics.com

Preprocessing Rules

For all the values in title/artist/genre/language/lyrics_clean, apply **in this order**:

1. **Lowercase** all text.
2. **Preserve commas and periods; remove all other punctuation/special characters.**
    - Keep: `,` and `.`
        i. Rationale: Keeping commas and periods preserves clause boundaries useful for later NLP tasks.
    - Remove: `! ? " ' : ; - _ ( ) [ ] { } / \ * # @ & % + = < > ^ | ~` and similar.
    - **Special symbols in words**: If symbols are used to stylize words (e.g., `LI$A`, `h@ppy`), the default rule is to **remove the symbol** (`LI$A` → `lia`). If you want, you may manually normalize them to the intended word (`LI$A` → `lisa`) for higher readability, but this is optional. In grading, both are acceptable as long as the rule is applied consistently.
3. Replace **all line breaks** with a single space.
4. Collapse **multiple spaces** into a single space.
5. **Do not** perform stemming or lemmatization.
6. Ensure the final file is **UTF-8** encoded (so non-English characters remain intact).

Reference Cleaner:

```python
import re

def clean_lyrics(text: str) -> str:
    """
    Clean lyrics text according to assignment rules:
    1. lowercase
    2. keep only: a-z, 0-9, space, comma, period
    3. replace line breaks with space
    4. collapse multiple spaces
    """
    # lowercase
    text = text.lower()
    # replace line breaks with space
    text = text.replace("\n", " ")
    # keep only allowed characters: letters, digits, spaces, commas, periods
    text = re.sub(r"[^a-z0-9\s,\.]", " ", text)
    # collapse multiple spaces into one
    text = re.sub(r"\s+", " ", text).strip()
    return text

# Example
raw_lyrics = """Money, money, money! $$$ LI$A sings @ night..."""
print("RAW:", raw_lyrics)
print("CLEANED:", clean_lyrics(raw_lyrics))
```

Validation Checklist (students must self-check before submitting)

- Exactly **50** lines, **no header**, UTF-8.
- Each line has **exactly 8 fields** separated by `||` in the required order.
- Dates are in `YYYY-MM-DD` and **≥ 2025-01-01**.
- `title/artist/genre/language/lyrics_clean` obeys the preprocessing rules (only lowercase letters/digits/spaces/ commas/periods).
- No duplicated *(title, artist)* pairs.
- Every line includes a valid lyrics **source URL**.

Submission

- Three files to upload:
  - The single `.txt` file on Canvas by the due date.
  - The Jupyter Notebook file (`.ipynb`)
  - `.pdf` of executed notebook (In Google Colab, click File > Print > Save as PDF). Sometimes, if you print too many values, the PDF will not contain the results later. It is your responsibility to make sure the PDF contains all the results you have. Below are a couple of online .ipynb to pdf converters to get the pdf file containing all the cells with output.
    - https://www.convert.ploomber.io/
    - https://www.vertopal.com/en/convert/ipynb-to-pdf

Grading Rubric (100%)

- **Completeness (40%)**: 50 songs with all required fields present, all three files submitted.
- **Correctness (30%)**: Accurate metadata, correct date formats, no duplicates.
- **Formatting (20%)**: Proper formatting, UTF-8 encoding.
- **Clarity (10%)**: Clean, readable, and well-organized dataset, clearly executable.
- For each of the dimensions (formatting, correctness, completeness, etc). If a student made 5 mistakes under one dimension, the student loses the whole point of this dimension. If only one mistake made, e.g. only one format issue like '|' instead of '||'. We just deduct 1/5 score of the format dimension.